**UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA**

Facultad de Ingeniería de Producción y Servicios

Escuela Profesional de Ingeniería de Sistemas

## Sistemas Operativos

Semestre VI – Periodo 2026-B

**PRÁCTICA DOMICILIARIA – SEMANA 8**

# Gestor seguro de recursos

Algoritmo del banquero, secuencias seguras y solicitudes dinámicas

**INFORME TÉCNICO**

**Modelo de múltiples procesos y tipos de recursos con simulación, validación y comparación entre evitación y detección**

**AUTOR**

**Gustavo Alonso Carrillo Villalta**

**DOCENTE**

**Norman Patrick Harvey Arce**

**PERIODO ACADÉMICO**

**2026-B**

Arequipa – Perú

**2026**

# 1. Resumen

Se desarrolla un gestor de recursos en Python 3 basado en el algoritmo del banquero. El sistema recibe el total de recursos, la matriz de demanda máxima y la asignación actual. Calcula `Available` y `Need`, simula solicitudes en copias del estado y solo concede una solicitud si el estado resultante conserva una secuencia segura. Las solicitudes inválidas o inseguras no modifican el estado real. Se incluyen pruebas para secuencia segura, solicitud superior a `Need`, recursos no disponibles, matrices inválidas, liberación y consistencia.

# 2. Modelo del sistema

Hay $N$ procesos y $M$ tipos de recursos. Las matrices contienen cantidades enteras no negativas:

- `Max[i][j]`: demanda máxima declarada del proceso $i$ para el recurso $j$.
- `Allocation[i][j]`: unidades actualmente asignadas.
- `Need`$[i][j] =$ `Max`$[i][j] -$ `Allocation`$[i][j]$.
- `Total[j]`: número total de instancias existentes.
- `Available`$[j] =$ `Total`$[j] - \sum_i$ `Allocation`$[i][j]$.

# 3. Invariantes y validación

1. $0 \leq Allocation[i][j] \leq Max[i][j]$.
2. $Need[i][j] = Max[i][j] - Allocation[i][j]$.
3. $Available[j] \geq 0$ y $Available[j] + \sum_i Allocation[i][j] = Total[j]$.
4. Toda fila tiene $M$ columnas y ambas matrices tienen $N$ filas.
5. Una solicitud se simula primero sobre copias. Si se rechaza o posterga, las matrices reales no cambian.

El constructor rechaza dimensiones incompatibles, cantidades negativas, asignaciones mayores que el máximo y totales insuficientes. El método de solicitud valida proceso, vector, límites de `Need` y `Available` antes de simular.

# 4. Algoritmo de seguridad

Se inicializa `Work = Available` y `Finish[i] = false`. Se busca un proceso no finalizado cuya necesidad sea menor o igual que `Work`. Se simula su terminación liberando su

asignación: `Work = Work + Allocation[i]` y `Finish[i] = true`. Se repite hasta no encontrar candidato. Si todos los valores de `Finish` son verdaderos, existe una secuencia segura.

```
Work = Available
Finish = [False] * N
while existe i con Finish[i] == False y Need[i] <= Work:
    Work = Work + Allocation[i]
    Finish[i] = True
seguro = all(Finish)
```

El programa devuelve, para cada paso, el proceso elegido, `Work` antes y después, la asignación liberada y el vector `Finish`. Así se puede auditar la secuencia y no solo recibir una etiqueta de seguro/inseguro.

## 5. Solicitud dinámica

Para una solicitud $Request_i$ se aplica el siguiente orden:

1. Si $Request_i > Need_i$, se rechaza por exceder la demanda declarada.
2. Si $Request_i > Available$, se posterga por falta de recursos libres.
3. Se simula la asignación en copias de `Available`, `Allocation` y `Need`.
4. Se ejecuta la prueba de seguridad sobre el estado simulado.
5. Si existe secuencia segura, se confirma el estado; en caso contrario, se rechaza y el estado original permanece intacto.

La distinción entre rechazar y postergar es importante: una solicitud puede ser legítima según `Need` pero no estar disponible en este momento. Una solicitud que cabe en `Available` puede ser rechazada si conduce a un estado inseguro.

# 6. Casos de prueba

| ID | Entrada | Resultado esperado |
|---|---|---|
| T1 | Estado clásico con cinco procesos y tres recursos | Secuencia segura $\langle P_1, P_3, P_0, P_2, P_4 \rangle$ usando índices desde cero. |
| T2 | Request superior a Need | Rechazo y estado sin cambios. |
| T3 | Request superior a Available pero dentro de Need | Postergación y estado sin cambios. |
| T4 | Matrices incompatibles o Allocation mayor que Max | Error de validación. |
| T5 | P4 solicita [3,3,0]: Request cabe en Need y Available | Rechazo porque el estado simulado no tiene secuencia segura. |
| T6 | Liberación válida de recursos | Aumenta Available y Need, disminuye Allocation. |
| T7 | Proceso inválido | Rechazo y estado sin cambios. |

## 6.1. Ejemplo clásico

Se usan los datos habituales de cinco procesos y tres tipos de recurso. El total es $(10, 5, 7)$; la asignación actual suma $(7, 2, 5)$, por lo que `Available` es $(3, 3, 2)$. La prueba de seguridad encuentra una secuencia de índices $[1, 3, 0, 2, 4]$, equivalente a $\langle P_1, P_3, P_0, P_2, P_4 \rangle$. En cada paso, la asignación del proceso finalizado vuelve a `Work`.

# 7. Evitación frente a detección

| Aspecto | Evitación (banquero) | Detección |
|---|---|---|
| Momento | Antes de confirmar cada solicitud | Después de permitir asignaciones, según la política de detección |
| Información | Requiere demanda máxima declarada y estado actual | Usa asignaciones actuales y solicitudes pendientes; no requiere que toda demanda máxima sea conocida |
| Decisión | No concede si el estado simulado es inseguro | Identifica procesos que no pueden finalizar con los recursos disponibles |
| Costo | Una prueba de seguridad por solicitud candidata | Costo periódico o bajo demanda, dependiente del algoritmo y del estado |
| Limitación | Puede aplazar solicitudes que serían completables en una evolución distinta; exige máximos fiables | Puede descubrir el problema después de que haya ocurrido y requiere política de recuperación |

En el modelo de múltiples instancias, la detección usa `Available`, `Allocation` y `Request` pendientes, simulando liberaciones de procesos que pueden completarse. No es idéntica a

la evitación: la evitación previene entrar en estados no seguros, mientras que la detección acepta el riesgo y busca identificar un bloqueo existente.

## 8. Pruebas y ejecución

```
python3 --version
python3 -m unittest -v
```

Las pruebas se encuentran en `test_banquero.py`. Se comprueba que las operaciones rechazadas no alteran el estado. El archivo `banquero.py` también puede ejecutarse desde un intérprete importando `ejemplo_clasico()`, llamando a `seguridad()` e inspeccionando el campo `pasos`.

## 9. Complejidad

En una prueba de seguridad se pueden buscar hasta $N$ procesos en cada iteración y realizar como máximo $N$ finalizaciones. Cada comparación vectorial cuesta $O(M)$; por tanto, una cota sencilla es $O(N^2M)$. Una solicitud dinámica añade la copia de matrices $O(NM)$ y una prueba de seguridad $O(N^2M)$, por lo que domina $O(N^2M)$. La memoria del modelo es $O(NM)$.

## 10. Limitaciones

El algoritmo supone que los máximos declarados son válidos y que los procesos liberan los recursos al finalizar. No representa prioridades, recursos no preemptibles con efectos externos, fallos de proceso ni costos de recuperación. Un estado inseguro no significa necesariamente que el sistema ya esté bloqueado: significa que no se puede garantizar la finalización para todas las demandas dentro del modelo.

## 11. Conclusiones

1. `Need` se calcula como diferencia entre máximo y asignación actual.
2. La existencia de recursos libres no basta: se debe probar la seguridad del estado resultante.
3. Simular sobre copias permite rechazar solicitudes sin corromper el estado original.
4. La evitación y la detección se distinguen por el momento de decisión y por la información requerida.
5. Mostrar `Work`, `Finish` y la asignación liberada permite justificar una secuencia segura paso a paso.

# 12. Referencias


1. A. Silberschatz, P. B. Galvin y G. Gagne, *Operating System Concepts*, 10.ª ed. Hoboken, NJ: Wiley, 2018.
2. A. S. Tanenbaum y H. Bos, *Modern Operating Systems*, 4.ª ed. Boston, MA: Pearson, 2015.
3. Python Software Foundation, “unittest — Unit testing framework”. Disponible en: https://docs.python.org/3/library/unittest.html.


# A. Código fuente completo

## A.1. Implementación: banquero.py

```python
from copy import deepcopy

class GestorBanquero:
    def __init__(self, maximo, allocation, total):
        self._validar_matrices(maximo, allocation, total)
        self.maximo = deepcopy(maximo)
        self.allocation = deepcopy(allocation)
        self.total = list(total)
        self.n = len(maximo)
        self.m = len(total)
        self.available = [self.total[j] - sum(self.allocation[i][j] for i in range(self.n))
                          for j in range(self.m)]
        self.need = [[self.maximo[i][j] - self.allocation[i][j] for j in range(self.m)]
                     for i in range(self.n)]

    @staticmethod
    def _validar_matrices(maximo, allocation, total):
        if not isinstance(total, list) or not total or any(type(x) is not int or x < 0 for x in total):
            raise ValueError("total debe ser una lista no vacia de enteros no negativos")
        if not isinstance(maximo, list) or not maximo or not isinstance(allocation, list):
            raise ValueError("matrices no vacias requeridas")
        if len(maximo) != len(allocation):
            raise ValueError("Max y Allocation deben tener igual numero de procesos")
        m = len(total)
        for matriz in (maximo, allocation):
            for fila in matriz:
                if not isinstance(fila, list) or len(fila) != m:
                    raise ValueError("dimensiones incompatibles")
                if any(type(x) is not int or x < 0 for x in fila):
                    raise ValueError("las matrices requieren enteros no negativos")
        for i in range(len(maximo)):
            for j in range(m):
                if allocation[i][j] > maximo[i][j]:
                    raise ValueError("Allocation no puede superar Max")
        for j in range(m):
            if sum(f[j] for f in allocation) > total[j]:
                raise ValueError("Allocation total supera recursos existentes")
            if any(maximo[i][j] > total[j] for i in range(len(maximo))):
                raise ValueError("Max de un proceso supera el total existente")

    def estado(self):
        return {"Total": self.total[:], "Available": self.available[:],
                "Max": deepcopy(self.maximo), "Allocation": deepcopy(self.allocation),
                "Need": deepcopy(self.need)}

    @staticmethod
    def _menor_igual(a, b):
```

```
        return all(x <= y for x, y in zip(a, b))

    def seguridad(self, available=None, allocation=None, need=None):
        work = list(self.available if available is None else available)
        alloc = self.allocation if allocation is None else allocation
        needs = self.need if need is None else need
        finish = [False] * self.n
        pasos = []
        secuencia = []
        while True:
            elegido = None
            for i in range(self.n):
                if not finish[i] and self._menor_igual(needs[i], work):
                    elegido = i
                    break
            if elegido is None:
                break
            antes = work[:]
            work = [work[j] + alloc[elegido][j] for j in range(self.m)]
            finish[elegido] = True
            secuencia.append(elegido)
            pasos.append({"proceso": elegido, "Work_antes": antes,
                          "Allocation_liberada": alloc[elegido][:],
                          "Work_despues": work[:], "Finish": finish[:]})
        return {"seguro": all(finish), "secuencia": secuencia,
                "Work_final": work, "Finish_final": finish, "pasos": pasos}

    def solicitar(self, proceso, request):
        # Todas las validaciones ocurren antes de mutar; las simulaciones usan copias.
        if type(proceso) is not int or not 0 <= proceso < self.n:
            return {"decision": "rechazada", "motivo": "proceso invalido"}
        if not isinstance(request, list) or len(request) != self.m or any(type(x) is not int or x < 0 for x in
 request):
            return {"decision": "rechazada", "motivo": "vector Request invalido"}
        if not self._menor_igual(request, self.need[proceso]):
            return {"decision": "rechazada", "motivo": "Request supera Need"}
        if not self._menor_igual(request, self.available):
            return {"decision": "postergada", "motivo": "recursos no disponibles"}
        av = [self.available[j] - request[j] for j in range(self.m)]
        al = deepcopy(self.allocation)
        ne = deepcopy(self.need)
        al[proceso] = [al[proceso][j] + request[j] for j in range(self.m)]
        ne[proceso] = [ne[proceso][j] - request[j] for j in range(self.m)]
        prueba = self.seguridad(av, al, ne)
        if not prueba["seguro"]:
            return {"decision": "rechazada", "motivo": "la asignacion produciria estado inseguro",
                    "prueba": prueba}
        self.available = av
        self.allocation = al
        self.need = ne
        return {"decision": "concedida", "motivo": "el estado resultante es seguro",
                "secuencia": prueba["secuencia"], "prueba": prueba}

    def liberar(self, proceso, unidades):
        if type(proceso) is not int or not 0 <= proceso < self.n:
            raise ValueError("proceso invalido")
        if not isinstance(unidades, list) or len(unidades) != self.m or any(type(x) is not int or x < 0 for x in
 unidades):
            raise ValueError("vector de liberacion invalido")
        if any(unidades[j] > self.allocation[proceso][j] for j in range(self.m)):
            raise ValueError("no se puede liberar mas de lo asignado")
        for j in range(self.m):
            self.allocation[proceso][j] -= unidades[j]
            self.available[j] += unidades[j]
            self.need[proceso][j] += unidades[j]

def ejemplo_clasico():
    maximo = [[7,5,3],[3,2,2],[9,0,2],[2,2,2],[4,3,3]]
    allocation = [[0,1,0],[2,0,0],[3,0,2],[2,1,1],[0,0,2]]
    return GestorBanquero(maximo, allocation, [10,5,7])

def ejemplo_solicitud_insegura():
```

```
    # Caso clasico: P1 solicita [3,3,0]; solicitud <= Need y <= Available,
    # pero la simulacion no conserva una secuencia segura.
    g = GestorBanquero(
        [[3,2],[1,2],[3,3]],
        [[1,0],[0,1],[1,1]],
        [3,3])
    return g
```

## A.2. Pruebas automatizadas: `test_banquero.py`

```
import unittest
from banquero import GestorBanquero, ejemplo_clasico, ejemplo_solicitud_insegura

class TestBanquero(unittest.TestCase):
    def test_estado_inicial_tiene_secuencia_segura(self):
        g = ejemplo_clasico()
        r = g.seguridad()
        self.assertTrue(r["seguro"])
        self.assertEqual(r["secuencia"], [1, 3, 0, 2, 4])

    def test_request_superior_a_need_rechazada_sin_mutacion(self):
        g = ejemplo_clasico()
        antes = g.estado()
        r = g.solicitar(1, [2, 0, 0])  # Need de P1 = [1,2,2]
        self.assertEqual(r["decision"], "rechazada")
        self.assertEqual(r["motivo"], "Request supera Need")
        self.assertEqual(g.estado(), antes)

    def test_request_superior_a_available_se_posterga(self):
        g = ejemplo_clasico()
        antes = g.estado()
        r = g.solicitar(0, [4, 0, 0])
        self.assertEqual(r["decision"], "postergada")
        self.assertEqual(g.estado(), antes)

    def test_solicitud_valida_pero_insegura(self):
        g = ejemplo_clasico()
        antes = g.estado()
        # P4: Request [3,3,0] <= Need [4,3,1] y <= Available [3,3,2],
        # pero el estado resultante no conserva una secuencia segura.
        r = g.solicitar(4, [3, 3, 0])
        self.assertEqual(r["decision"], "rechazada")
        self.assertEqual(r["motivo"], "la asignacion produciria estado inseguro")
        self.assertEqual(g.estado(), antes)

    def test_datos_invalidos_no_alteran_estado(self):
        g = ejemplo_clasico()
        antes = g.estado()
        r = g.solicitar(99, [1, 0, 0])
        self.assertEqual(r["decision"], "rechazada")
        self.assertEqual(g.estado(), antes)

    def test_matrices_invalidas(self):
        with self.assertRaises(ValueError):
            GestorBanquero([[2, 1]], [[3, 0]], [3, 2])

    def test_liberacion_actualiza_need_y_available(self):
        g = ejemplo_clasico()
        g.liberar(0, [0, 1, 0])
        self.assertEqual(g.available, [3, 4, 2])
        self.assertEqual(g.need[0], [7, 5, 3])

if __name__ == "__main__":
    unittest.main(verbosity=2)
```